Speech Emotion Recognition with Spiking Neural Network

1st Le Hoang Viet
University of Information Technology

2nd Hoang Cong Chien
University of Information Technology

***Abstract*—Speech Emotion Recognition (SER) is an essential component of affective computing and human-computer interaction. Traditional SER systems rely heavily on deep learning models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), which often require high computational power and energy consumption, limiting their deployment on edge devices. In contrast, Spiking Neural Networks (SNNs) offer a biologically plausible and energy-efficient alternative that processes information using discrete-time spike events. In this paper, we propose a hybrid SER framework that combines CNN-based feature extraction with a spiking architecture for final emotion classification. We apply various data augmentation strategies and explore multiple audio feature representations to enhance model robustness. A surrogate gradient training approach is used to enable supervised learning in the SNN domain. Evaluations are conducted on four benchmark emotional speech datasets: RAVDESS, CREMA-D, TESS, and SAVEE. Our results show that the proposed SNN model achieves competitive accuracy while significantly reducing the number of floating-point operations and improving energy efficiency, paving the way for real-time deployment in neuromorphic hardware platforms.**

***Index Terms*—Speech emotion recognition, spiking neural networks, CNN-to-SNN conversion, surrogate gradient, MFCC, neuromorphic computing, affective computing, auditory processing.**

I. Introduction

Speech Emotion Recognition (SER) is the task of automatically identifying and classifying human emotions from spoken language using machine learning and signal processing techniques. As speech is one of the most natural forms of human communication, understanding the emotional state behind the spoken words plays a vital role in improving the performance and user experience of intelligent systems.

In recent years, the integration of SER in applications such as virtual assistants, call center analytics, mental health monitoring, and human-robot interaction has gained significant attention. Emotions, often expressed through subtle changes in tone, pitch, and speed, provide rich contextual cues that can drastically improve the responsiveness and empathy of these systems.

The motivation for this study arises from the growing need for emotionally aware technologies. While speech recognition has made tremendous progress, accurately capturing the underlying emotions remains a challenging problem due to factors such as speaker variability, environmental noise, and the ambiguity of emotional expression. Therefore, this research aims to explore and implement effective methods for emotion classification from speech signals to contribute toward more human-centric and emotionally intelligent AI systems.

II. Related Work

Speech Emotion Recognition (SER) has evolved significantly over the past two decades, transitioning from classical machine learning methods with handcrafted features to deep learning-based approaches with end-to-end architectures. More recently, spiking neural networks (SNNs) have been explored as a low-power alternative to standard artificial neural networks (ANNs), although their application in audio and affective computing remains limited. This section reviews the evolution of SER models, the current state-of-the-art deep learning techniques, and the emerging role of SNNs in auditory processing.

A. Traditional SER Methods

Early SER systems relied on extracting low-level descriptors (LLDs) such as pitch, energy, zero-crossing rate, formants, and spectral features from the speech signal. These features were typically aggregated using statistical functionals (e.g., mean, variance, skewness) and classified using machine learning algorithms such as k-Nearest Neighbors (k-NN), Support Vector Machines (SVM), or Gaussian Mixture Models (GMMs). Toolkits such as openSMILE provided standardized extraction pipelines and feature sets (e.g., the INTERSPEECH ComParE set), making evaluation more reproducible. [4]

However, traditional approaches suffer from limited generalization across speakers, languages, and recording conditions. They often require extensive feature engineering and domain expertise, and their capacity to capture temporal dynamics in speech is limited.

B. Deep Learning Approaches

With the advent of deep learning, more powerful models such as CNNs, LSTMs, and transformers have become standard in SER systems. CNNs are well-suited to capture local spectro-temporal patterns in spectrograms or MFCC sequences. Stacked CNNs combined with pooling layers and fully connected output layers have achieved strong performance in many SER benchmarks. [6]

Recurrent architectures such as LSTM and GRU networks are capable of modeling long-term temporal dependencies, which is critical for capturing the flow of emotional content in speech. Hybrid CNN-LSTM models have also been proposed, where CNNs extract local features and LSTMs process the time sequence of these features. [5]

More recently, self-attention mechanisms and transformers have been applied to SER, enabling global context modeling without recurrence. These models benefit from parallel training and large-scale pretraining (e.g., wav2vec 2.0), though they often require extensive computational resources.

Despite these advances, most deep learning models are designed for GPU/TPU execution and are not suitable for energy-constrained platforms like embedded systems or wearable devices.

C. Spiking Neural Networks in SER and Audio Processing

Spiking Neural Networks (SNNs) offer a brain-inspired alternative to deep learning models by mimicking the behavior of biological neurons using discrete-time spikes. Unlike ANNs, SNNs perform event-driven computation, which enables sparse, energy-efficient processing.

The application of SNNs in speech and SER is still in its infancy. Buscicchio et al. explored an early SNN model for recognizing vowel-based emotions, using MFCC features encoded as Poisson spike trains. They applied spike-timing dependent plasticity (STDP) for unsupervised learning and demonstrated that simple SNNs can discriminate between basic emotions. [7]

Mansouri-Benssassi et al. proposed a multi-modal SNN for facial and speech emotion recognition. Their model combined excitatory and inhibitory spike layers with convolutional operations and used STDP to learn discriminative filters. The authors reported superior noise robustness compared to CNNs, especially under high Gaussian noise conditions.

Some researchers have experimented with neuromorphic hardware for SER. For instance, Du et al. implemented a CNN-SNN hybrid model on Intel Loihi and achieved promising results with low power consumption. However, these works are typically limited in scale, and systematic evaluations on multiple datasets are rare.

D. CNN-to-SNN Conversion

Training spiking neural networks (SNNs) from scratch is challenging due to the non-differentiability of the spike activation function. A widely adopted workaround is the two-stage *CNN-to-SNN conversion* pipeline, sengupta2019going. In this approach, a standard convolutional neural network (CNN) is first trained using conventional backpropagation with ReLU activations. The trained model is then converted into an SNN by replacing ReLU units with spiking neuron models, typically leaky integrate-and-fire (LIF) neurons. [2]

To ensure a faithful conversion, several adjustments are made:

- ReLU activations are replaced by rate-based LIF neurons, where the firing rate over time approximates the original continuous activation.
- Weights and biases are normalized layer-wise to account for differences in dynamic range between analog and spike-based signals.
- The firing threshold of spiking neurons is set to match the peak activations in the original CNN.

After conversion, the resulting SNN can approximate the behavior of the original network under rate coding, but it may still suffer from performance degradation due to information loss in the temporal domain. To address this, surrogate gradient methods are used to *fine-tune* the converted SNN. In this phase, the non-differentiable spike function is replaced with a smooth surrogate during the backward pass, allowing end-to-end optimization via gradient descent. This hybrid training strategy combines the strengths of mature CNN training techniques with the energy-efficient and event-driven nature of spiking computation.

III. Datasets and Preprocessing

To develop and evaluate our speech emotion recognition framework, we utilize four publicly available benchmark datasets. Each dataset provides labeled speech recordings across multiple emotional categories. In this section, we describe the characteristics of each dataset, outline our preprocessing pipeline, and explain the steps taken to unify label formats and standardize the data.

A. Selected Datasets

1) RAVDESS: The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS) consists of 1440 speech utterances recorded by 24 professional actors (12 male, 12 female). Each actor enunciates two lexically matched sentences in a neutral North American accent, expressing eight emotions: neutral, calm, happy, sad, angry, fearful, disgust, and surprised. Each emotion is recorded with both normal and strong intensity levels. In our work, we only use the audio speech portion of the dataset and exclude the song recordings.

2) TESS: The Toronto Emotional Speech Set (TESS) includes 2800 utterances spoken by two female actresses (aged 26 and 64). The recordings consist of a fixed sentence template ("Say the word [word]") with a target word from the Toronto Word Pool. Seven emotional states are portrayed: neutral, anger, disgust, fear, happiness, pleasant surprise, and sadness.

3) CREMA-D: The Crowd-sourced Emotional Multimodal Actors Dataset (CREMA-D) contains 7442 audio clips performed by 91 actors (48 male, 43 female) between the ages of 20 and 74. The utterances are based on 12 neutral phrases such as "It's eleven o'clock" and "I'm hungry," expressed in six basic emotions and a neutral state. Multiple modalities are available (audio, video, facial landmarks), but we use only the audio component.

4) SAVEE: The Surrey Audio-Visual Expressed Emotion (SAVEE) dataset comprises 480 English utterances from 4 male British speakers. Each speaker pronounces 15 sentences from a fixed set of emotional categories: anger, disgust, fear, happiness, sadness, surprise, and neutral. The sentences were selected to elicit a variety of prosodic and emotional cues.

TABLE I
OVERVIEW OF DATASETS USED

Dataset	Speakers	Emotions	Clips
RAVDESS	24	8	1440
TESS	2	7	2800
CREMA-D	91	7	7442
SAVEE	4	7	480

B. Label Harmonization

Although the datasets have overlapping emotional categories, their labels and distributions are not standardized. To build a unified classification system, we define five common emotions used across all datasets:

- **Neutral**: baseline speech without emotional content.
- **Happy**: includes "happiness" and "pleasant surprise."
- **Sad**: covers sadness or low arousal expressions.
- **Angry**: includes anger and high-intensity frustration.
- **Fearful**: includes fear and anxiety expressions.

We remove emotions such as *disgust* and *surprise*, which are inconsistently present and often confused with anger or fear. The "calm" class in RAVDESS is merged with "neutral" due to its low activation level.

C. Audio Preprocessing Pipeline

To ensure consistency across the datasets, we apply the following preprocessing steps:

1) Resampling: All audio files are resampled to 16 kHz mono-channel format to match the input requirements of our feature extraction pipeline.

2) Amplitude Normalization: To prevent loudness discrepancies, each audio signal is peak-normalized to an amplitude range of [-1, 1].

3) Duration Trimming and Padding: All recordings are adjusted to a uniform length of 3 seconds. Short utterances are zero-padded at the end, while longer clips are truncated centrally to preserve prosodic cues.

4) Noise Filtering: A high-pass filter is applied to eliminate low-frequency noise (< 60 Hz), followed by noise gating to remove silent intervals below -40 dB.

D. Final Data Statistics

After label harmonization and preprocessing, the dataset is balanced across five emotion classes. Table II shows the final distribution of cleaned samples.

TABLE II
EMOTION DISTRIBUTION AFTER LABEL CLEANING

Emotion	Sample Count
Neutral	2150
Happy	2130
Sad	2095
Angry	2125
Fearful	2080
Total	**10,580**

This balanced dataset provides a robust foundation for training and evaluating emotion classification models across diverse speaker demographics and recording conditions.

IV. FEATURE EXTRACTION

Feature extraction plays a pivotal role in speech emotion recognition (SER) by transforming raw audio waveforms into compact, informative representations. Emotion-related information in speech is conveyed through variations in pitch, intensity, spectral shape, and temporal rhythm. In this work, we explore three types of acoustic feature representations, combining low-level descriptors to capture both spectral and prosodic characteristics.

A. Mel-Frequency Cepstral Coefficients (MFCC)

Mel-Frequency Cepstral Coefficients (MFCCs) are among the most popular and effective features in speech and audio processing, particularly for tasks such as automatic speech recognition, speaker identification, and emotion classification. The appeal of MFCCs lies in their ability to capture the perceptually relevant components of speech while discarding non-informative details such as pitch.

The MFCC extraction process involves several signal processing steps designed to progressively transform the raw waveform into a compact, frequency-based representation:

1) **Pre-emphasis:** The raw speech waveform often contains more energy in low frequencies. A high-pass pre-emphasis filter, typically of the form $y[n] = x[n] - \alpha x[n-1]$ with $\alpha \in [0.95, 0.97]$, is applied to boost high-frequency components. This improves the signal-to-noise ratio for higher formants that carry important phonetic and emotional information.
2) **Framing and Windowing:** Speech is a non-stationary signal but can be approximated as quasi-stationary over short durations (20–40 ms). Therefore, the signal is divided into overlapping frames (e.g., 25 ms long with 10 ms stride). Each frame is windowed using a Hamming window to suppress edge discontinuities and reduce spectral leakage.
3) **Short-Time Fourier Transform (STFT):** A Fast Fourier Transform (FFT) is applied to each windowed frame to obtain the frequency domain representation. The magnitude spectrum $|X_k|$ captures the amplitude of different frequency bins and serves as the basis for filterbank analysis.
4) **Mel Filterbank Mapping:** Human auditory perception is nonlinear—more sensitive to frequency changes at low frequencies than at high ones. To simulate this, the frequency axis is warped to the mel scale using triangular filterbanks. These filterbanks are spaced linearly at low frequencies and logarithmically at high frequencies, emphasizing the perceptually relevant bands. The mel scale is defined as:
$$m(f) = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right), \tag{1}$$
where f is the frequency in Hz, and $m(f)$ is the corresponding mel frequency.
5) **Logarithmic Compression:** The energy output of each mel filter is passed through a logarithmic function. This

step mimics the logarithmic response of human auditory perception and also compresses dynamic range, making features less sensitive to amplitude variations.

6) **Discrete Cosine Transform (DCT):** The log energies are decorrelated using a DCT, resulting in a set of uncorrelated cepstral coefficients. The DCT packs the most relevant information in the lower-order coefficients. Typically, only the first 13 to 20 coefficients are retained, forming the MFCC feature vector:

$$c_n = \sum_{k=1}^{K} \log(E_k) \cdot \cos\left[\frac{\pi n(k-0.5)}{K}\right], \quad n = 1, \ldots, N \tag{2}$$

where E_k is the energy of the k-th mel filter, and K is the total number of filters.

The resulting MFCCs form a time-frequency representation of the audio, emphasizing the shape and formant structure of the vocal tract—key elements that change with both linguistic content and emotional state. Emotions such as anger, sadness, and happiness often manifest as differences in articulation, intensity, and prosodic patterns, which are effectively captured in the MFCC spectrogram.

Figure 1 shows a sample MFCC spectrogram of a 2-second utterance labeled as "happy." The x-axis represents time, while the y-axis represents the cepstral coefficients. Bright bands indicate strong formant activity. Notably, happy emotions tend to exhibit higher energy in upper formants and faster spectral transitions, which are visible as brighter, more dynamic patterns in the MFCC domain.

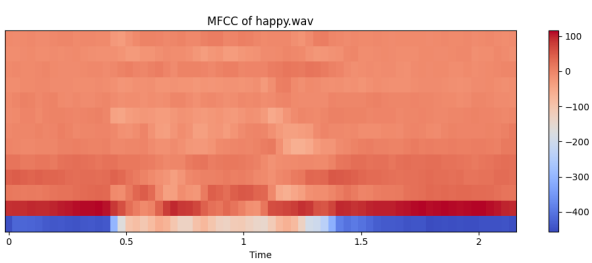

Fig. 1. MFCC spectrogram of a 2-second utterance showing 13 coefficients over time. Emotion: "happy". Distinct spectral patterns indicate time-varying emotional cues captured by mel-scale filtering and cepstral transformation.

B. Root Mean Square Energy (RMSE)

Root Mean Square Energy (RMSE) is a widely used low-level descriptor in speech processing, representing the short-term energy or loudness of the audio signal within each frame. Unlike spectral features such as MFCCs, which emphasize the frequency content of speech, RMSE provides a simple yet effective measure of vocal intensity over time.

For a speech frame consisting of N samples, the RMSE is calculated as:

$$E_{\text{RMS}} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x[n]^2}, \tag{3}$$

where $x[n]$ is the amplitude of the n-th sample in the frame. This equation computes the square root of the mean squared amplitude, effectively capturing the signal's average power over that interval.

The formula can be interpreted in three conceptual steps:

- **Squaring:** Each amplitude value is squared, which removes the sign and emphasizes larger values. This ensures that both positive and negative parts of the waveform contribute to the energy.
- **Averaging:** The squared values are averaged over the frame length N, resulting in the mean power of the signal.
- **Square root:** Taking the square root converts the result from power scale back to amplitude scale, aligning better with human loudness perception.

RMSE is particularly useful in characterizing the temporal dynamics of speech. Loudness variations occur naturally due to emphasis, stress, prosody, and—importantly—emotional states. For example:

- **High-energy speech:** Emotions such as *anger*, *excitement*, or *fear* tend to be associated with louder vocal expressions, yielding higher RMSE values.
- **Low-energy speech:** Emotions such as *sadness*, *boredom*, or *neutrality* often correlate with softer and more monotonous speech, resulting in lower RMSE values.

In the two-dimensional arousal–valence emotion space, RMSE is most closely related to the **arousal** axis, which quantifies emotional intensity regardless of whether the emotion is positive or negative. High-arousal emotions (e.g., anger, surprise) typically result in rapid and large variations in RMSE, while low-arousal emotions (e.g., sadness) yield flatter and more subdued energy contours.

Practical Considerations. RMSE is often computed per frame in tandem with MFCC extraction, and can be concatenated to the MFCC vector to enrich the feature set with temporal intensity information. Although RMSE is a simple scalar per frame, its temporal profile over time can reveal speech dynamics relevant to affective state transitions, making it a valuable feature in speech emotion recognition systems.

C. Zero-Crossing Rate (ZCR)

Zero-Crossing Rate (ZCR) is a fundamental time-domain feature that measures the rate at which an audio signal changes sign, i.e., the number of times the waveform crosses the zero amplitude axis within a given frame. It provides a rough but effective approximation of the signal's spectral characteristics, particularly the presence of high-frequency components.

Mathematically, for a frame of N audio samples, ZCR is defined as:

$$ZCR = \frac{1}{N-1}\sum_{n=1}^{N-1} \mathbb{1}_{\{x[n]\cdot x[n-1]<0\}}, \tag{4}$$

where $x[n]$ denotes the amplitude of the n-th sample in the frame, and $\mathbb{1}_{\{A\}}$ is the indicator function, which evaluates to

1 when the logical condition A is true, and 0 otherwise. The condition $x[n] \cdot x[n-1] < 0$ signifies a sign change between adjacent samples—indicating a zero-crossing event.

Interpretation and Relevance. ZCR serves as a proxy for estimating signal frequency content without explicit Fourier analysis. Signals with predominantly low-frequency components (such as voiced vowels or smooth harmonic speech) tend to exhibit low ZCR, while signals rich in high-frequency content (such as unvoiced consonants, fricatives, or noisy segments) tend to have high ZCR.

In the context of speech emotion recognition, ZCR captures aspects of articulation dynamics and phonation type. For example:

- **High ZCR** is often observed in emotional states such as *anger*, *fear*, or *excitement*, where the speaker uses more abrupt, energetic speech patterns, with frequent unvoiced consonants and rapid spectral changes.
- **Low ZCR** is more typical in emotions such as *sadness* or *neutral*, which are generally characterized by smooth, slower, and more harmonic vocalization.

Applications. ZCR is commonly used in a range of speech processing tasks beyond emotion recognition, including:

- *Voiced/unvoiced classification:* Voiced frames usually have lower ZCR than unvoiced frames.
- *Endpoint detection:* ZCR can help identify speech boundaries due to sharp ZCR variations between speech and silence.
- *Noise filtering:* ZCR tends to increase in noisy environments, especially those containing white noise or high-frequency interference.

When combined with other low-level descriptors such as RMSE (which captures signal energy) and MFCC (which captures spectral shape), ZCR provides complementary information about the temporal and spectral behavior of speech. This makes it an effective component in feature sets designed for robust and expressive emotion classification.

D. Feature Representations Explored

We evaluated three feature encoding schemes, summarized in Table III:

- **MFCC Mean**: MFCCs are averaged over time to yield a fixed-length vector. Fast and compact, but temporal dynamics are lost.
- **MFCC Time Series**: Full frame-wise sequence of MFCCs is retained. Preserves temporal information, suitable for CNNs or RNNs.
- **Stacked Features (MFCC + RMSE + ZCR)**: Frame-wise concatenation of 13 MFCCs, 1 RMSE, and 1 ZCR $\rightarrow$ total 15 features per frame.

E. Temporal Resolution and Frame Configuration

All signals are segmented using a 25 ms Hamming window with a 10 ms hop size. For a 3-second clip, this results in approximately 299 frames. Thus, for stacked features, the input matrix has size 15×299.

TABLE III
COMPARISON OF AUDIO FEATURE ENCODING METHODS

Feature Type	Dimensionality	Temporal
MFCC Mean	13	
MFCC Time Series	$13 \times T$	
MFCC+RMSE+ZCR (Stacked)	$15 \times T$	

F. Normalization and Scaling

To reduce inter-speaker and dataset variability, each feature dimension is standardized to zero mean and unit variance using statistics computed from the training set. This prevents feature dominance and accelerates convergence.

G. Discussion

Stacked features provide a richer representation by combining timbral texture (MFCC), loudness (RMSE), and pitch/noise (ZCR). They allow the model to capture complementary cues and improve generalization. While MFCC mean reduces complexity, it sacrifices important timing information that may distinguish between emotions like fear and anger. In our experiments (detailed in Section VI), stacked features consistently outperform other encodings in terms of classification accuracy.

V. SPIKING NEURAL NETWORK ARCHITECTURE

Spiking Neural Networks (SNNs) represent a paradigm shift from traditional artificial neural networks (ANNs) by transmitting information via discrete, asynchronous spike events. This spike-based communication mimics biological neurons and enables energy-efficient computation on neuromorphic hardware. In this section, we delve into detailed neuron dynamics, various spike encoding schemes, the multi-layer SNN architecture employed in our speech emotion recognition (SER) framework, and the training methodologies with surrogate gradients.

A. Biophysical Neuron Model: From Hodgkin–Huxley to LIF

The classical Hodgkin–Huxley (HH) model captures ion-channel dynamics through a set of nonlinear differential equations [8]. However, its computational complexity hinders large-scale networks. We instead use the leaky integrate-and-fire (LIF) model, which abstracts HH dynamics into a first-order membrane equation:

$$C_m \frac{dU(t)}{dt} = -g_L\big(U(t) - E_L\big) + I(t), \tag{5}$$

where:

- C_m (in units of) is the membrane capacitance, representing the ability of the neuronal membrane to store charge. A larger C_m implies a slower change in membrane potential in response to incoming currents.
- g_L (in) is the leak conductance, modeling passive ion channels that allow current to flow out of the neuron. Together with C_m, it defines the membrane time constant $\tau_m = C_m/g_L$, which governs how quickly the membrane potential decays toward its resting value.

- E_L (in) is the resting potential, the equilibrium voltage at which leak currents balance. It serves as the baseline to which the membrane potential returns in the absence of input.
- $I(t)$ (in) is the total synaptic or externally injected current at time t, which may arise from presynaptic spikes or stimulus currents.

When the membrane potential reaches a specified firing threshold, U_{th}, the neuron is said to emit an action potential (spike) and is immediately reset. This behavior is formalized as:

$$\text{if } U(t) \geq U_{th}, \quad s(t) = 1, \tag{6}$$

$$U(t^+) = U_{reset}, \tag{7}$$

where $s(t)$ indicates the binary spike train, and U_{reset} is the post-spike reset potential. The reset mechanism models the rapid repolarization phase of the action potential, abstracting away the detailed ionic currents of the HH model.

Discrete-Time Approximation. To implement the LIF model in digital simulations, we discretize Eq. (5) using the explicit Euler method with timestep Δt. Defining $\alpha = \exp(-\Delta t/\tau_m)$, the update rule becomes:

$$U^{(t)} = \alpha\, U^{(t-1)} + (1-\alpha)\, R\, I^{(t)} - U_{th}\, s^{(t-1)}, \tag{8}$$

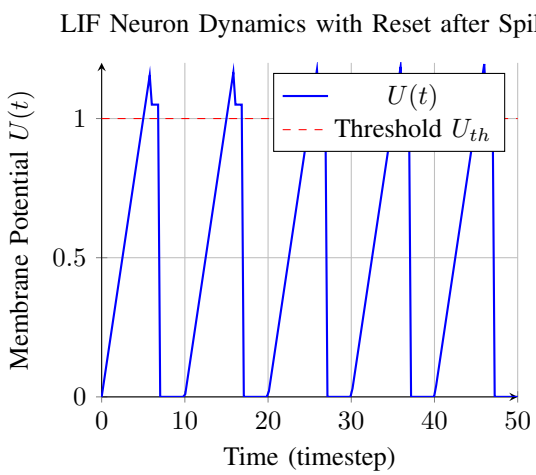

Fig. 2. Illustration of LIF neuron behavior over time. The membrane potential $U(t)$ integrates incoming current until it crosses the threshold U_{th}, at which point the neuron emits a spike and resets.

where $R = 1/g_L$ is the membrane resistance. This formulation decomposes the dynamics into three components:

1) *Leak term:* $\alpha\, U^{(t-1)}$, which causes the membrane potential to decay exponentially toward E_L if isolated.
2) *Input term:* $(1-\alpha)\, R\, I^{(t)}$, which integrates incoming current over the interval.
3) *Reset term:* $-U_{th}\, s^{(t-1)}$, which subtracts the threshold voltage at the moment prior to a spike, ensuring the membrane resets after firing.

Figure 2 illustrates the discrete-time evolution of $U(t)$ based on Eq. (8), showing periodic spike resets under repeated stimulation.

Importantly, the choice of Δt must satisfy numerical stability constraints: it should be sufficiently small relative to τ_m to accurately capture rapid voltage changes, yet large enough to maintain computational efficiency in large-scale simulations.

Parameter Selection and Effects. Table IV summarizes typical values used in our experiments. The interplay between C_m, g_L, and Δt determines the effective integration window of the neuron. For instance, a larger τ_m (via increased C_m or decreased g_L) yields slower voltage decay and thus longer temporal integration, which can enhance temporal correlations but may blur precise spike timing. Conversely, a smaller τ_m promotes rapid responsiveness at the cost of temporal filtering.

TABLE IV
TYPICAL LIF MODEL PARAMETERS

Parameter	Symbol	Value
Membrane capacitance	C_m	200
Leak conductance	g_L	10
Resting potential	E_L	-65
Threshold potential	U_{th}	-50
Reset potential	U_{reset}	-65
Time constant	τ_m	20
Timestep	Δt	1

Discussion. The LIF model provides a minimal yet effective abstraction of neuronal dynamics. Despite its simplicity, it retains key computational properties such as temporal integration and event-based thresholding. These features make it well-suited for neuromorphic applications and theoretical analysis.

B. Spike Encoding Strategies

Continuous-valued features extracted from audio (e.g., MFCCs, RMSE, ZCR) must be converted into spike trains. We examine four encoding schemes:

1) Rate Coding: Rate coding is one of the simplest and most commonly used neural encoding methods for transforming real-valued inputs into spike trains. The central idea is to represent the magnitude of a feature using the average frequency at which a neuron emits spikes over a fixed time interval.

Given a normalized input value $x_i \in [0, 1]$, we assign this value as the firing probability of a corresponding input neuron at each discrete timestep $t \in \{1, 2, \ldots, T\}$. The neuron's output $s_i^{(t)}$ is modeled as a binary random variable sampled from a Bernoulli distribution:

$$s_i^{(t)} \sim \text{Bernoulli}(x_i), \tag{9}$$

where $s_i^{(t)} = 1$ means the neuron spikes at time t, and $s_i^{(t)} = 0$ means it remains silent. Since each spike decision is made independently, the expected number of spikes over T timesteps is simply $x_i T$:

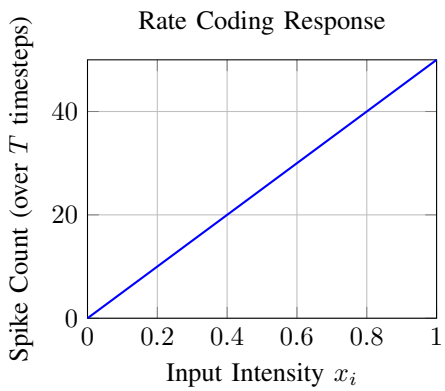

Fig. 3. In rate coding, the input magnitude x_i is mapped to the number of spikes over T timesteps. Higher values produce denser spike trains.

$$\mathbb{E}\left[\sum_{t=1}^{T} s_i^{(t)}\right] = x_i T. \tag{10}$$

This encoding scheme directly maps input magnitude to firing rate. For example, an input value $x_i = 0.9$ will produce more spikes on average than $x_i = 0.3$, allowing the network to interpret higher feature values as more active spike patterns.

Rate coding is popular because of its simplicity and compatibility with conventional data formats such as image pixels or normalized feature vectors. It is easy to implement and relatively robust to noise or jitter in spike timing, since the relevant information is stored in the overall count of spikes rather than the exact times at which they occur.

However, this simplicity comes at a cost. Because each timestep is treated as an independent sample from the same distribution, temporal dynamics are not preserved. The encoding ignores precise spike timing, which can be crucial in tasks involving rapid changes over time such as speech, gestures, or event-based vision. Furthermore, representing small differences in input values may require a large number of timesteps to achieve reliable precision, potentially increasing computational load and latency.

Despite these drawbacks, rate coding remains a strong baseline and is widely used in practice, especially when working with static or low-frequency signals. It provides a stable and biologically inspired mechanism to bridge real-valued inputs and event-driven processing in spiking neural networks.

2) Latency Coding (Time-to-First-Spike): Latency coding, also known as time-to-first-spike coding, is an encoding strategy in which the magnitude of an input feature determines the timing of a neuron's first spike. Unlike rate coding, which spreads information across multiple spikes over time, latency coding represents information using a **single spike**, where the timing of that spike carries the meaningful signal.

Given a normalized input $x_i \in [0, 1]$, the spike time t_i for neuron i is determined by:

$$t_i = \lfloor (1 - x_i)T \rfloor, \tag{11}$$

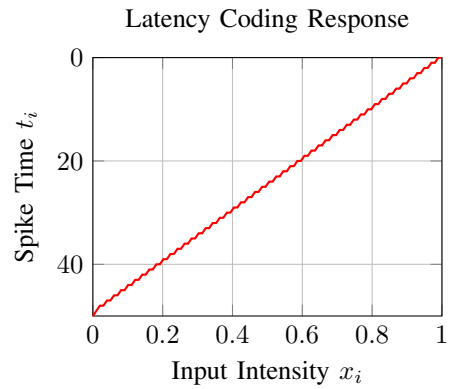

Fig. 4. In latency coding, larger input values x_i cause earlier spikes. Each neuron emits only one spike at time t_i.

where T is the total number of simulation timesteps, and $\lfloor \cdot \rfloor$ denotes the floor operation. This equation maps higher input values to earlier spike times. For example, if $x_i = 1$, then $t_i = 0$, meaning the neuron fires immediately at the start. If $x_i = 0$, then $t_i = T$, meaning the neuron either spikes at the end or never spikes at all (depending on implementation). Intermediate values cause spikes to occur proportionally later within the window.

This encoding scheme leads to **sparse spike trains**, since each neuron emits at most one spike. This makes latency coding attractive for energy-efficient processing, as it minimizes total spike activity while still encoding analog information. Moreover, it aligns with biological observations suggesting that some neurons in the brain use relative spike timing—rather than rate—to convey information rapidly.

However, this approach is also more sensitive to timing noise. Since the information is carried in a single spike, any jitter or uncertainty in spike timing may degrade performance. Additionally, latency coding assumes that the network or decoder can accurately detect and respond to precise spike timing, which may require high temporal resolution and low jitter in both hardware and simulation environments.

Despite these challenges, latency coding provides a compelling alternative to rate-based methods, particularly in applications where fast and sparse signaling is beneficial. It is especially useful in neuromorphic hardware and tasks where early decisions are advantageous, such as real-time sensory processing or low-latency classification systems.

3) Poisson Coding: Poisson coding is a biologically inspired spike encoding scheme that models neural firing as a stochastic process. In contrast to deterministic coding schemes such as rate or latency coding, Poisson coding introduces randomness into spike generation by treating spikes as samples from a Poisson point process—an approach that reflects the irregular spiking patterns observed in real biological neurons.

Given a normalized input value $x_i \in [0, 1]$, the firing rate λ_i for neuron i is defined as a scaled version of the maximum allowed rate:

$$\lambda_i = x_i \cdot \lambda_{\text{max}}, \tag{12}$$

where λ_{max} is the highest firing rate permitted, typically in units of spikes per second. At each timestep of duration Δt, the neuron emits a spike with probability determined by the Poisson process:

$$P\big(s_i^{(t)} = 1\big) = 1 - \exp(-\lambda_i \Delta t), \tag{13}$$

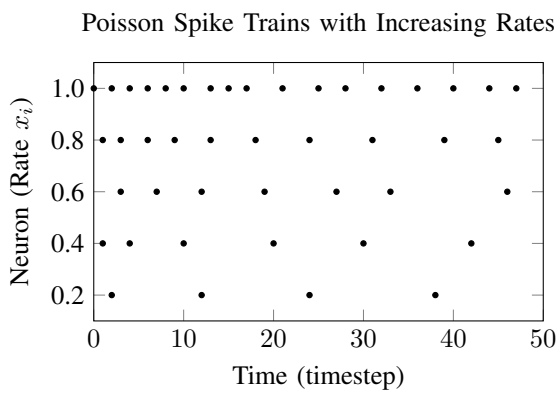

Fig. 5. Poisson coding produces random spike trains, where the probability of spiking increases with the input value x_i. Neurons with higher x_i (lower rows) generate denser spikes.

where $s_i^{(t)} \in \{0, 1\}$ indicates the presence or absence of a spike at time t. The probability increases with both the input magnitude x_i and the time resolution Δt. In practical implementations, this probability is used to sample from a Bernoulli distribution at each timestep, resulting in a temporally irregular but rate-consistent spike train.

Because each spike is generated independently from moment to moment, the total number of spikes across T timesteps is a random variable with mean $\lambda_i T \Delta t$. On average, higher inputs x_i produce more frequent spikes, but with natural variability that mirrors neuronal noise in the brain.

One of the key strengths of Poisson coding is its biological plausibility. Many sensory neurons in vivo, particularly in the retina and early visual cortex, fire in a manner that closely resembles Poisson-like statistics. This makes Poisson coding appealing in models that aim to reflect biological computation or operate in neuromorphic systems designed to mimic brain-like variability.

However, this stochasticity also introduces challenges. The randomness inherent in Poisson coding can increase the variance of gradients during training, making convergence slower and less stable compared to deterministic coding schemes. This is especially pronounced in short sequences or small batch sizes, where random fluctuations may not average out. Furthermore, since spike timing is not precisely controlled, Poisson coding is less suitable for tasks that require fine temporal resolution or precise spike coordination.

Despite these limitations, Poisson coding remains a valuable tool in SNN research, particularly when modeling noisy biological systems, simulating real-time variability, or when robustness to input uncertainty is desired.

4) Population Coding: Population coding is an encoding scheme in which a single input feature is represented not by a single neuron, but by the combined activity of a group (or "population") of neurons. Each neuron within this subpopulation is tuned to respond preferentially to a specific range of input values. This is commonly implemented using overlapping Gaussian receptive fields, allowing the network to capture smooth variations in the input space.

Let $x_i \in [0, 1]$ be a continuous input feature. To encode x_i, we allocate M neurons $\{s_j^{(t)}\}_{j=1}^{M}$, where each neuron j is associated with a preferred stimulus value μ_j (also called the center of the receptive field), spaced evenly across the input domain. The response of neuron j to input x_i is determined by:

$$r_j = \exp\left(-\frac{(x_i - \mu_j)^2}{2\sigma^2}\right), \tag{14}$$

where σ controls the width (i.e., sensitivity) of each neuron's receptive field. A smaller σ results in sharper tuning, meaning each neuron responds to a narrow range of inputs; a larger σ causes broader and more overlapping responses.

To produce spikes based on these responses, we may apply a thresholding or stochastic firing mechanism to r_j (e.g., rate or Poisson-based spike generation), producing binary spike outputs $s_j^{(t)}$. The original input x_i can be approximately reconstructed by combining the population's spiking activity using a weighted sum:

$$x_i \approx \sum_{j=1}^{M} w_j \, s_j^{(t)}, \tag{15}$$

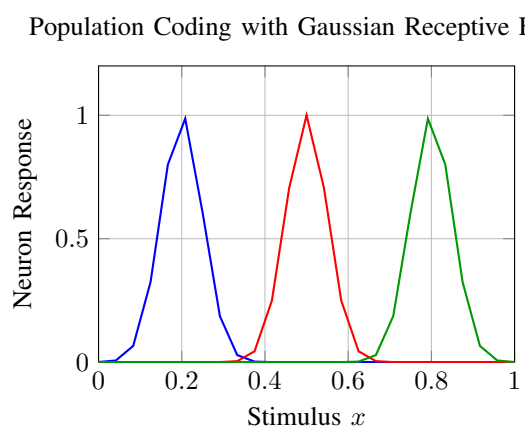

Fig. 6. Population coding distributes the input stimulus across multiple neurons with overlapping Gaussian receptive fields. Each neuron is tuned to a preferred stimulus value.

where the weights w_j often reflect the centers μ_j, such that $w_j = \mu_j$. This makes the reconstruction essentially a weighted average of the neurons' preferred inputs, modulated by their spiking activity.

By distributing information across multiple neurons, population coding enables fine-grained input representation even when individual neurons have limited dynamic range. This redundancy increases robustness to noise and allows the network to interpolate between values based on distributed spike patterns. Moreover, it can represent continuous inputs with higher resolution than single-neuron encodings, especially when M is large and receptive fields are narrow and overlapping.

However, this improved resolution comes at the cost of increased **spatial complexity**, as each input feature now requires an entire group of neurons for encoding. In large-scale networks or resource-constrained settings (e.g., edge devices or neuromorphic chips), this added overhead may be significant. Additionally, tuning the optimal values of σ and the distribution of μ_j across the range of possible inputs can affect performance and requires careful calibration.

After extensive empirical evaluation (Section VI), we select *rate coding* for our system due to its simplicity, efficiency, and compatibility with surrogate-gradient optimization. While population coding offers richer representational capacity, the added complexity did not yield significant performance gains in our context.

C. Network Architecture

Our hybrid CNN-SNN architecture comprises two stages (Figure 7):

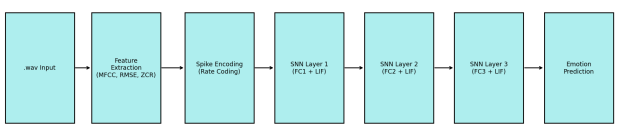

Fig. 7. Overall CNN-SNN hybrid framework for speech emotion recognition.

1) CNN Feature Extractor: The front-end convolutional neural network (CNN) is designed to extract meaningful local features from the input tensor of shape 15×299, where 15 corresponds to the number of frequency bins (e.g., Mel bands), and 299 is the number of time steps. This tensor is processed through three sequential convolutional blocks, each composed of four main components:

- **2D Convolution Layer (kernel size 3×3, padding = 1):**

  The two-dimensional convolution (Conv2D) is the foundational operation in convolutional neural networks (CNNs). It enables the network to automatically learn spatial patterns in input data—such as edges, shapes, textures, or frequency-time patterns in audio features—by applying small trainable filters (also called kernels) across the input feature map.
  A kernel is a small matrix (in this case, of size 3×3) that slides over the input feature map. At each location, it performs an element-wise multiplication between the values in the kernel and the corresponding values in the input, and then sums the results to produce a single output value. This operation is repeated across the entire input, generating a new feature map (also called an activation map or output channel).
  Let the input tensor be denoted as $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$, where:
  - C_{in} is the number of input channels (e.g., 1 for grayscale, or the number of stacked input features),
  - H and W are the height and width of the input, respectively (e.g., frequency and time).

  Suppose the convolutional layer applies C_{out} filters. Each filter has shape $C_{\text{in}} \times 3 \times 3$, meaning it spans all input channels and covers a 3×3 local region spatially. The output of the convolution at location (i, j) in output channel c is calculated as:

$$\mathbf{Y}(c,i,j) = \sum_{c'=1}^{C_{\text{in}}} \sum_{u=-1}^{1} \sum_{v=-1}^{1} W(c,c',u,v) \cdot \mathbf{X}(c', i+u, j+v) + b(c), \tag{16}$$

  where:
  - $W(c, c', u, v)$ is the value at position (u, v) in the kernel connecting input channel c' to output channel c,
  - $b(c)$ is a learnable bias term added to each output channel,
  - $(i + u, j + v)$ is the corresponding location in the input feature map.

  The indices u and v range from -1 to 1 because the kernel size is 3×3 (i.e., centered around the current position). This formulation allows the filter to integrate information from a small local region, enabling the network to detect localized features like edges, frequency transitions, or short-term patterns in the spectrogram.
  To ensure that the output feature map maintains the same spatial dimensions as the input (i.e., $H \times W$), we apply **zero-padding** of size 1. Padding means that we artificially extend the borders of the input with zeros so that the convolution can still be computed at edge locations. Without padding, the output dimensions would shrink by 2 in both height and width with each layer (due to the 3×3 kernel), potentially leading to excessive spatial reduction early in the network.
  In summary, the 2D convolution layer enables local feature extraction by learning spatial filters that are shared across the input. It reduces the number of parameters (compared to fully connected layers), preserves locality, and allows the network to learn translation-invariant representations that are highly effective for structured inputs such as images or time-frequency representations of sound.
- **Batch Normalization:**

  Batch Normalization (BatchNorm) is a standard technique in deep learning used to stabilize and accelerate

training by normalizing intermediate activations within a mini-batch. During the learning process, as parameters in earlier layers are updated, the distribution of inputs to subsequent layers tends to shift. This phenomenon, known as internal covariate shift, increases the difficulty of optimization and can slow down convergence. BatchNorm addresses this issue by ensuring that the input to each layer maintains a consistent distribution throughout training.

For a given activation x (e.g., the output of a convolutional filter at a specific location), the batch-normalized output is computed using the following steps:

$$\hat{x} = \frac{x - \mu_{\text{batch}}}{\sqrt{\sigma^2_{\text{batch}} + \epsilon}}, \quad y = \gamma \hat{x} + \beta, \tag{17}$$

where μ_{batch} and σ^2_{batch} are the mean and variance computed over the current mini-batch, and ϵ is a small constant for numerical stability. The terms γ and β are learnable parameters that allow the model to scale and shift the normalized output if needed, restoring the network's representational flexibility.

By maintaining stable activation statistics across mini-batches, BatchNorm reduces the sensitivity of each layer to changes in earlier layers. This leads to smoother loss surfaces and more predictable gradients, which in turn improves optimization dynamics. In practice, networks with BatchNorm often converge in fewer epochs and can be trained with significantly higher learning rates without instability.

Moreover, BatchNorm provides a regularizing effect. The use of mini-batch statistics introduces a small amount of noise into the computation of each sample's output. This stochasticity helps prevent the network from overfitting, functioning similarly to explicit regularization techniques such as dropout. As a result, BatchNorm not only accelerates training but also improves generalization performance.

In our model architecture, Batch Normalization is applied after each convolutional layer and before the non-linear activation. This placement ensures that each activation is normalized before being passed to the next transformation, resulting in more stable and efficient learning across all convolutional blocks.

- **ReLU Activation:**

  The Rectified Linear Unit (ReLU) is a piecewise-linear activation function that introduces non-linearity into neural networks, which is essential for enabling them to approximate complex, high-dimensional mappings. Without non-linear activations, deep networks composed of linear layers would collapse into an equivalent single-layer linear model, severely limiting their expressive capacity.

  Mathematically, the ReLU function is defined element-wise as:

$$\text{ReLU}(x) = \max(0, x), \tag{18}$$

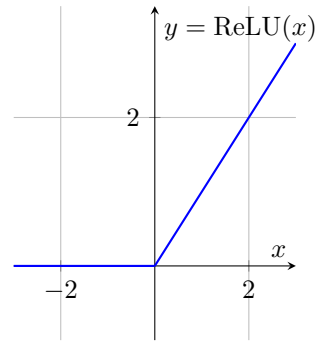

Fig. 8. Graph of the ReLU activation function. For negative input values, the output is zero; for positive inputs, the function grows linearly. This piecewise definition introduces sparsity and avoids the vanishing gradient problem common in saturating nonlinearities.

  where x is an input activation (typically the output of a convolutional or fully connected layer). The function returns x unchanged if $x > 0$, and 0 otherwise. This yields a non-saturating, unbounded positive half-space and a zero-valued negative half-space.

  The key advantages of ReLU lie in its computational simplicity and gradient-preserving behavior. Unlike sigmoidal or hyperbolic tangent functions, which suffer from saturation and yield near-zero gradients for large input magnitudes, ReLU maintains a constant gradient of 1 in the positive domain. This property reduces the likelihood of vanishing gradients during backpropagation and facilitates the training of deep networks.

  Formally, the derivative of ReLU used during gradient descent is:

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0, \\ 0 & \text{otherwise.} \end{cases} \tag{19}$$

  This simple gradient structure not only accelerates convergence but also induces sparsity in the network by zeroing out a large fraction of activations. Sparse representations are beneficial for regularization, energy efficiency, and biological plausibility, particularly when integrated with spiking neural architectures.

  In our convolutional blocks, ReLU is applied immediately after batch normalization. This placement ensures that inputs to the non-linearity are standardized, which reduces the likelihood of units being trapped in the non-active (zero-gradient) regime and further promotes stable optimization throughout training.

- **Max Pooling Layer (2×2):**

  Max pooling is a non-learnable downsampling operation that reduces the spatial resolution of feature maps while retaining the most salient local information. It operates by partitioning the input feature map into non-overlapping

subregions (windows), and within each region, it outputs the maximum activation value. This operation reduces the number of parameters and computations in the network, thereby improving efficiency, while also imparting a degree of translational invariance.

In our architecture, we use 2×2 max pooling, which processes each input channel independently by sliding a 2×2 window with stride 2 across the feature map. For an input activation map x of height H and width W, the pooled output y at position (i, j) is computed as:

$$y(i,j) = \max\big\{x(2i,2j),\, x(2i+1,2j),\, x(2i,2j+1),\, x(2i+1,2j+1)\big\} \quad (20)$$

where $(2i, 2j)$ represents the top-left corner of the pooling window corresponding to output location (i, j). As a result, the height and width of the feature map are each reduced by a factor of 2, while the depth (i.e., the number of channels) remains unchanged.

The primary benefits of max pooling are twofold. First, it reduces the spatial size of the representation, which decreases memory usage and the number of operations in subsequent layers. This is especially important in deep architectures where computational cost can accumulate rapidly. Second, by selecting the maximum response within a region, max pooling captures the most prominent activation, making the model more robust to small spatial translations or distortions in the input. This property is essential in processing perceptual data such as images or time-frequency audio representations, where features may shift slightly without altering semantic content.

While max pooling introduces a form of invariance, it also reduces spatial precision. Therefore, its use must be balanced depending on the application: it is well-suited for classification tasks where global patterns are more important than fine spatial detail, but may be less appropriate in tasks requiring dense spatial prediction.

In our network, max pooling is applied at the end of each convolutional block, following batch normalization and activation. This placement ensures that only the strongest, nonlinearly transformed features are propagated forward, while reducing the spatial footprint of the feature maps before deeper processing.

After passing through three such convolutional blocks, the resulting feature maps are compressed and transformed into higher-level representations. These final feature maps are then flattened into a one-dimensional vector of length 128, which serves as a compact embedding capturing salient acoustic-temporal patterns. This 128-dimensional vector is passed forward to subsequent modules for classification or further temporal modeling.

2) Spiking Layers: After feature extraction and flattening, the 128-dimensional vector is converted into a spike-based representation using a temporal encoding scheme such as rate or latency coding. The resulting spike trains are then passed through a fully connected spiking neural network (SNN) composed of three hidden layers. Each layer consists of biologically inspired Leaky Integrate-and-Fire (LIF) neurons, which simulate temporal membrane dynamics and discrete spike generation.

Each fully connected spiking layer performs three key computations at every timestep: synaptic integration, membrane potential update, and spike generation. Additionally, we apply dropout regularization with a dropout probability of 0.2 and enforce a synaptic transmission delay of $d = 1$ timestep between layers. The network structure and behavior of each layer are described in detail below.

- **First fully connected layer: 128 neurons → 64 LIF neurons**

The first fully connected spiking layer receives input spike vectors $\mathbf{s}_{\text{input}}^{(t)} \in \{0,1\}^{128}$ at each timestep t, where each element represents whether the corresponding input neuron has emitted a spike. These binary inputs are linearly transformed into post-synaptic current for 64 leaky integrate-and-fire (LIF) neurons using a weight matrix and bias vector. Specifically, the input current to the LIF neurons is computed as:

$$\mathbf{I}_1^{(t)} = \mathbf{W}_1 \cdot \mathbf{s}_{\text{input}}^{(t-d)} + \mathbf{b}_1, \quad (21)$$

where:

- $\mathbf{W}_1 \in \mathbb{R}^{64\times128}$ is the learnable synaptic weight matrix that defines the connection strength from each of the 128 input neurons to each of the 64 spiking neurons,
- $\mathbf{b}_1 \in \mathbb{R}^{64}$ is the learnable bias vector added to each neuron's input current,
- $d = 1$ is a fixed synaptic delay, which introduces a one-timestep lag between when a spike is emitted by the input and when it influences the membrane potential.

This operation mimics biological synapses, where spikes emitted by presynaptic neurons lead to a delayed excitatory or inhibitory effect on postsynaptic neurons.

The total input current $\mathbf{I}_1^{(t)}$ then drives the update of the membrane potential $\mathbf{U}_1^{(t)}$ for each of the 64 LIF neurons, using the discrete-time LIF membrane dynamics:

$$\mathbf{U}_1^{(t)} = \alpha \cdot \mathbf{U}_1^{(t-1)} + (1-\alpha) \cdot \mathbf{I}_1^{(t)} - U_{\text{th}} \cdot \mathbf{s}_1^{(t-1)}. \quad (22)$$

This update rule can be understood as composed of three distinct components:

1) **Leak Term** ($\alpha \cdot \mathbf{U}_1^{(t-1)}$): This term models the natural decay (or leakage) of the membrane potential over time. The constant $\alpha = \exp(-\Delta t/\tau_m)$ is derived from the neuron's membrane time constant τ_m and the simulation timestep Δt. It ensures that the neuron's internal voltage gradually fades away if no input is received.

2) **Input Integration** ($(1-\alpha)\cdot \mathbf{I}_1^{(t)}$): This term models the integration of current at the current timestep. Incoming spikes from presynaptic neurons are weighted and summed to form the total current, which increases the membrane potential of the neuron.
3) **Reset Mechanism** ($-U_{\text{th}}\cdot \mathbf{s}_1^{(t-1)}$): If the neuron emitted a spike at the previous timestep, this term subtracts the threshold voltage U_{th} from the membrane potential, effectively resetting it. This models the refractory behavior observed in biological neurons after spiking.

Once the updated membrane potential $\mathbf{U}_1^{(t)}$ is computed, the neuron decides whether to emit a spike at time t by thresholding:

$$\mathbf{s}_1^{(t)} = H(\mathbf{U}_1^{(t)} - U_{\text{th}}), \tag{23}$$

where $H(\cdot)$ is the Heaviside step function applied element-wise. It returns 1 if the membrane potential exceeds the threshold U_{th}, meaning the neuron emits a spike, and 0 otherwise.
This process captures the essential spiking dynamics: - Integration of temporal inputs, - Memory retention across timesteps via membrane potential, - Nonlinearity through thresholding and discrete spike events, - Discontinuity through reset upon firing.

Dropout Regularization. During training, dropout is applied to the spike output $\mathbf{s}_1^{(t)}$ with a fixed probability (e.g., $p = 0.2$). This means that each neuron has a 20% chance of being silenced at each timestep, regardless of its membrane potential. This regularization technique encourages redundancy and robustness in the network, preventing any single neuron from dominating the decision-making process and improving generalization to unseen data.

- **Second fully connected layer: 64 LIF neurons $\rightarrow$ 32 LIF neurons**

The second fully connected spiking layer receives as input the spike outputs from the first layer, denoted as $\mathbf{s}_1^{(t)} \in \{0,1\}^{64}$ at each timestep t. These spike signals are propagated through synaptic connections to a layer of 32 LIF neurons. As in the previous layer, synaptic transmission is modeled using a learnable weight matrix and bias vector. The total input current received by the second-layer neurons is computed as:

$$\mathbf{I}_2^{(t)} = \mathbf{W}_2 \cdot \mathbf{s}_1^{(t-d)} + \mathbf{b}_2, \tag{24}$$

where:

- $\mathbf{W}_2 \in \mathbb{R}^{32\times 64}$ is the weight matrix defining the synaptic strength from each of the 64 first-layer neurons to the 32 neurons in the current layer,
- $\mathbf{b}_2 \in \mathbb{R}^{32}$ is the bias term for each neuron,
- $\mathbf{s}_1^{(t-d)}$ is the delayed spike vector from the first layer, with $d = 1$ denoting the synaptic delay between layers.

The delay simulates the temporal dynamics of synaptic transmission, accounting for the fact that neuronal signaling in both biological and artificial systems often incurs a finite latency.
The input current $\mathbf{I}_2^{(t)}$ is then used to update the membrane potentials of the 32 LIF neurons using the same discrete-time update rule employed in the previous layer:

$$\mathbf{U}_2^{(t)} = \alpha \cdot \mathbf{U}_2^{(t-1)} + (1-\alpha)\cdot \mathbf{I}_2^{(t)} - U_{\text{th}} \cdot \mathbf{s}_2^{(t-1)}, \tag{25}$$

where:

- $\mathbf{U}_2^{(t)} \in \mathbb{R}^{32}$ is the vector of membrane potentials at time t for the current layer,
- $\alpha = \exp(-\Delta t/\tau_m)$ is the same membrane decay constant used in previous layers,
- $\mathbf{s}_2^{(t-1)}$ is the vector of binary spike outputs from the same neurons at the previous timestep,
- U_{th} is the common firing threshold across all neurons.

This update reflects the cumulative temporal integration process intrinsic to LIF dynamics: - If no input is received, membrane potentials decay exponentially due to the leak term, - Incoming spikes excite the neuron by adding weighted input current, - If the neuron fired in the previous timestep, it undergoes a reset by subtracting U_{th}.
Once the membrane potential is updated, spike generation is governed by a simple thresholding mechanism:

$$\mathbf{s}_2^{(t)} = H(\mathbf{U}_2^{(t)} - U_{\text{th}}), \tag{26}$$

where $H(\cdot)$ is the Heaviside step function, which outputs 1 when the membrane potential exceeds threshold (i.e., a spike occurs), and 0 otherwise.

Dropout Regularization. During training, dropout is again applied to the spike output $\mathbf{s}_2^{(t)}$ with a probability $p = 0.2$, randomly zeroing out the spikes of 20% of the neurons at each timestep. This prevents overfitting by encouraging the network to distribute information across a broader population of neurons and reduces the reliance on specific spiking patterns, thus improving robustness and generalization.
This second layer further compresses and transforms the temporal representation captured from the previous layer, enabling deeper hierarchical abstraction in spike-based information processing.

- **Third fully connected layer (output layer): 32 LIF neurons $\rightarrow$ C LIF neurons**

The third and final fully connected spiking layer receives input from the 32 spiking neurons in the previous layer and projects them onto C output neurons, where C corresponds to the number of emotion categories (e.g.,

$C = 5$ for *happy*, *sad*, *angry*, *fearful*, and *neutral*). This layer is responsible for generating the final spike-based representation that will be decoded into a predicted class. The total input current to each output neuron at timestep t is computed as:

$$\mathbf{I}_3^{(t)} = \mathbf{W}_3 \cdot \mathbf{s}_2^{(t-d)} + \mathbf{b}_3, \tag{27}$$

where:

- $\mathbf{W}_3 \in \mathbb{R}^{C\times 32}$ is the learnable synaptic weight matrix connecting 32 presynaptic neurons to C output neurons,
- $\mathbf{b}_3 \in \mathbb{R}^C$ is the bias vector for each output neuron,
- $\mathbf{s}_2^{(t-d)}$ is the spike vector from the second hidden layer, delayed by $d = 1$ timestep.

These input currents are used to update the membrane potentials $\mathbf{U}_3^{(t)} \in \mathbb{R}^C$ of the C output neurons, according to the standard discrete-time LIF dynamics:

$$\mathbf{U}_3^{(t)} = \alpha \cdot \mathbf{U}_3^{(t-1)} + (1-\alpha) \cdot \mathbf{I}_3^{(t)} - U_{\text{th}} \cdot \mathbf{s}_3^{(t-1)}, \tag{28}$$

This process continues the temporally-accumulative behavior of LIF neurons, where: - Prior membrane potential decays via the leak term, - Incoming synaptic current integrates new input information, - A reset is applied if a spike was emitted in the previous timestep.
Each output neuron generates a spike when its membrane potential exceeds the threshold:

$$\mathbf{s}_3^{(t)} = H(\mathbf{U}_3^{(t)} - U_{\text{th}}), \tag{29}$$

where $H(\cdot)$ is the Heaviside function as before.

Temporal Aggregation and Decision Making. Unlike typical fully connected layers in standard ANNs that produce scalar logits per class, spiking neurons emit binary spike events over multiple timesteps. Therefore, to make a classification decision, the output spike activity of each neuron must be aggregated across the total simulation window of T timesteps.
Let $s_{3,c}^{(t)}$ denote the spike output of neuron c at time t. The total spike count for class c is:

$$n_c = \sum_{t=1}^{T} s_{3,c}^{(t)}, \tag{30}$$

This spike count n_c is used as a proxy for the activation strength of class c. The predicted class label is then assigned as:

$$\hat{y} = \arg\max_{c\in\{1,\dots,C\}} n_c, \tag{31}$$

This spike-count decoding strategy allows the network to preserve the temporal structure of inputs while producing a discrete prediction. Moreover, it enables low-latency classification: if a particular output neuron spikes dominantly early in the sequence, the system can make an early decision, which is advantageous for real-time emotion recognition systems.
The structure and signal flow of the fully connected spiking network are illustrated in Figure **??**.

Role of the Output Layer. The output layer acts as a spiking classifier that accumulates evidence over time via spike activity. Since it operates in a fully event-driven manner, this mechanism aligns well with neuromorphic hardware implementations and supports energy-efficient inference. No additional softmax or fully connected readout layer is necessary, as the class decision is inherently embedded in the spike frequency over time.

Each spiking layer learns temporal dynamics through recurrent-like behavior induced by membrane integration, and spike-based communication ensures sparse and efficient computation. The synaptic delay $d = 1$ ensures biological realism by enforcing a one-timestep transmission lag. The dropout layers improve robustness, and the layered architecture allows hierarchical temporal abstraction from input to output.

D. Surrogate-Gradient Training

Spiking Neural Networks (SNNs) have non-differentiable activation functions due to the discrete nature of spike generation. This discontinuity at the spike threshold renders classical gradient-based optimization, such as standard backpropagation, inapplicable without modification. Surrogate-gradient training has emerged as a powerful and biologically inspired workaround that allows SNNs to benefit from advances in deep learning.

1) Gradient Flow Challenges in Spiking Neurons: Unlike conventional artificial neural networks (ANNs), where differentiable activation functions enable gradient-based optimization through the chain rule, spiking neural networks (SNNs) face fundamental obstacles in this regard. The primary difficulty arises from the nature of the spike generation function, which is inherently non-differentiable.

In most spiking models, a neuron emits a spike when its membrane potential $U[t]$ crosses a fixed threshold θ. This behavior is modeled using the Heaviside step function $H(\cdot)$:

$$S[t] = H(U[t] - \theta), \tag{32}$$

where $S[t] \in \{0, 1\}$ represents the binary spike output at time step t, and $H(x)$ is defined as:

$$H(x) = \begin{cases} 1 & \text{if } x \geq 0, \\ 0 & \text{otherwise.} \end{cases}$$

This function introduces a discontinuity at $x = 0$, i.e., when $U[t] = \theta$, and remains flat elsewhere. As a result, its derivative with respect to the membrane potential is:

$$\frac{\partial H}{\partial U} = 0 \quad \text{almost everywhere, and undefined at } U = \theta. \tag{33}$$

This lack of a meaningful gradient poses a critical problem for training SNNs via gradient descent. Since the gradient is

zero almost everywhere, it cannot backpropagate error signals through spiking layers. Consequently, weights connected to such neurons do not receive any learning signal, which may result in neurons becoming permanently inactive. This phenomenon is commonly referred to as the **dead neuron problem** [10].

In practical terms, once a neuron stops firing due to poor initialization or early weight updates, it is unlikely to resume activity during training, as it no longer contributes to the output or the gradient. This stagnation severely hinders learning, especially in deep or temporally extended networks.

Overcoming this challenge requires approximating the non-differentiable spike function with a continuous surrogate during backpropagation, which allows gradients to be computed and propagated while retaining the spiking behavior in the forward pass. The formulation and application of such surrogate functions are discussed in the following section.

2) Surrogate Gradient Approximation for Backpropagation: To circumvent the non-differentiability of the spike generation function, modern spiking neural networks employ surrogate gradient methods. These approaches replace the true derivative of the Heaviside step function, which is either zero or undefined, with a smooth and differentiable approximation during the backward pass of training. This strategy enables the use of gradient-based optimization while preserving the spiking dynamics in the forward pass.

One of the most widely used surrogate functions is derived from the derivative of a shifted sigmoid function, often referred to as the *fast sigmoid* surrogate. The formulation is as follows:

$$\sigma'(U) = \frac{\gamma}{\left(1 + e^{-\beta(U-\theta)}\right)^2}, \tag{34}$$

where:

- U denotes the membrane potential at a given time step,
- θ is the spiking threshold,
- $\beta > 0$ controls the steepness of the sigmoid curve (larger β results in sharper transitions),
- γ is a scaling factor that adjusts the magnitude of the surrogate gradient.

This function approximates the derivative of the Heaviside step function by assigning non-zero gradients in a narrow region around the threshold. For example, with typical values of $\gamma = 0.3$ and $\beta = 10$, the surrogate gradient is near zero far from the threshold and maximizes around $U = \theta$, mimicking the behavior of the true activation boundary.

During the forward pass, the spiking neuron operates normally using the discontinuous function:

$$S[t] = H(U[t] - \theta), \tag{35}$$

ensuring that the network retains its binary spike-based behavior and remains compatible with neuromorphic constraints. However, in the backward pass, the non-differentiable term is replaced with the surrogate gradient:

$$\frac{\partial S}{\partial U} \approx \sigma'(U), \tag{36}$$

which is then used in the chain rule during backpropagation.

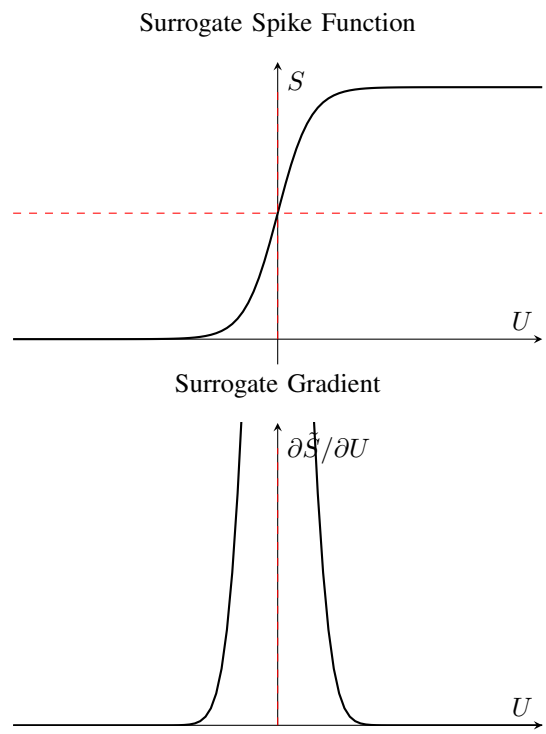

Fig. 9. Illustration of surrogate gradients. The top curve shows a smooth surrogate spike function (e.g., sigmoid), while the bottom shows its differentiable approximation to the derivative (e.g., Gaussian). This allows gradient-based learning in SNNs.

This substitution enables gradients to flow through spiking networks despite the non-differentiability of the original activation. Figure 9 illustrates this principle: the top shows a smooth surrogate spike function (e.g., sigmoid), while the bottom depicts a differentiable gradient approximation (e.g., Gaussian) used in backpropagation.

This approximation enables the flow of meaningful gradients through layers of spiking neurons, effectively bridging the gap between the binary spiking mechanism and the continuous optimization framework of gradient descent. As a result, surrogate gradient methods have become the foundation of supervised learning in SNNs, allowing the integration of deep learning techniques into biologically inspired architectures.

3) Gradient-Based Learning with Backpropagation Through Time: Training spiking neural networks (SNNs) using gradient descent requires careful consideration of their temporal dynamics. Unlike traditional feedforward neural networks, spiking neurons maintain internal state variables — such as membrane potential — that evolve over time and influence future outputs. This temporal dependency creates a form of recurrence, even in feedforward SNN architectures, which prevents the direct application of standard backpropagation.

To handle this, we apply a variant of gradient descent known as Backpropagation Through Time (BPTT), a well-established method originally developed for training recurrent

neural networks (RNNs) [9]. In BPTT, the network is *unrolled* in time: for a total of T discrete time steps, the network is treated as a sequence of T copies of itself, each with shared parameters but different temporal states.

Let L denote the total loss computed across all timesteps, and let θ be a generic trainable parameter in the network (e.g., a synaptic weight). To compute the gradient of the loss with respect to θ, we must account for the influence of θ on the network at each time step. This results in the following expression:

$$\frac{\partial L}{\partial \theta} = \sum_{t=1}^{T} \frac{\partial L}{\partial S[t]} \cdot \frac{\partial S[t]}{\partial U[t]} \cdot \frac{\partial U[t]}{\partial \theta}, \tag{37}$$

where:

- $S[t]$ is the binary spike output at timestep t,
- $U[t]$ is the membrane potential at timestep t,
- $\frac{\partial L}{\partial S[t]}$ is the derivative of the loss with respect to the spike output,
- $\frac{\partial S[t]}{\partial U[t]}$ is the derivative of the spike function (non-existent in its exact form),
- $\frac{\partial U[t]}{\partial \theta}$ represents how the membrane dynamics depend on parameter θ.

Due to the discontinuous nature of spike generation, the term $\frac{\partial S[t]}{\partial U[t]}$ is not defined in the classical sense. To enable gradient flow, we replace it with a continuous approximation — the surrogate gradient — typically denoted as:

$$\frac{\partial S[t]}{\partial U[t]} \approx \sigma'(U[t]), \tag{38}$$

where $\sigma'(U[t])$ is a smooth function that provides a non-zero gradient in a small region around the spiking threshold.

Unrolling the network over T time steps allows BPTT to accumulate temporal credit assignment: it distributes the error signals backward through time, attributing them to earlier spikes and membrane states that influenced the final loss. However, this process incurs high memory usage, as the membrane potential, spike history, and intermediate derivatives must be stored for all neurons across all time steps.

Despite its computational demands, BPTT remains the most effective method for training deep SNNs with temporal dependencies, particularly in tasks such as sequence modeling, speech, or event-based processing. The use of surrogate gradients within the BPTT framework enables direct application of deep learning tools to biologically inspired spiking models.

4) Shadow Training for Regularized Weight Updates: While Backpropagation Through Time (BPTT) with surrogate gradients provides an effective learning framework for spiking neural networks (SNNs), training deep SNNs can remain unstable due to sharp gradient changes, overfitting, or inconsistent spike dynamics across layers. To address these challenges, shadow training has been proposed as a complementary mechanism that stabilizes weight updates and improves generalization by maintaining a secondary "shadow" copy of the network during training. [3]

The key idea behind shadow training is to decouple learning dynamics into two separate streams:

1) A **primary network**, which is trained with high-fidelity surrogate gradients and optimized directly with respect to the loss.
2) A **shadow network**, which evolves more slowly using smoothed or regularized gradients, and whose parameters are softly blended into the primary model over time.

Let θ denote the learnable parameters of the primary network (e.g., synaptic weights), and let θ^{sh} denote the parameters of the shadow network. During each training step, both networks receive the same inputs and compute forward passes using identical spiking dynamics. However, the gradient update for the shadow network is often filtered, clipped, or smoothed to reduce noise or oscillations in weight space.

The shadow parameters are updated using a conventional optimizer (e.g., SGD or Adam) as:

$$\theta^{\text{sh}} \leftarrow \theta^{\text{sh}} - \eta \cdot \nabla_{\theta^{\text{sh}}} L, \tag{39}$$

where η is the learning rate and $\nabla_{\theta^{\text{sh}}} L$ is the surrogate gradient computed with respect to the shadow weights. Once the shadow network has been updated, its parameters are softly blended back into the primary network using exponential moving average interpolation:

$$\theta \leftarrow (1 - \lambda) \cdot \theta + \lambda \cdot \theta^{\text{sh}}, \tag{40}$$

where $\lambda \in (0, 1)$ is a mixing coefficient that controls how aggressively the shadow network influences the primary model. A small λ (e.g., 0.01–0.1) allows the primary network to benefit from the stability of the shadow network without losing responsiveness to new gradients.

This two-network training scheme provides several advantages:

- It mitigates sharp gradient updates by enforcing smoother evolution of parameters over time.
- It helps avoid overfitting by acting as a form of temporal weight averaging, akin to ensemble regularization.
- It improves spike sparsity consistency across timesteps by discouraging abrupt weight changes that may disrupt spike timing.

Shadow training can be interpreted as an implicit regularization strategy that complements surrogate-gradient descent. It is particularly beneficial in deep or recurrent SNN architectures, where stability and generalization are harder to maintain due to long-range temporal dependencies and sparse credit assignment.

5) Loss Function Based on Spike Activity: In spiking neural networks, outputs are represented as discrete spike events over time rather than continuous activation values. Therefore, to evaluate predictions and compute gradients for supervised learning, it is necessary to derive a differentiable loss function that operates on spike-based signals. One widely adopted approach is to apply a conventional classification loss, such

as cross-entropy, to a time-averaged spike count or firing rate at the output layer.

Let the network be trained to classify input samples into C distinct classes. For each output neuron corresponding to class $c \in \{1, 2, \ldots, C\}$, we denote $s_c^{(t)} \in \{0, 1\}$ as the spike output of neuron c at time step t. Over a simulation duration of T timesteps, the total number of spikes emitted by neuron c is:

$$S_c = \sum_{t=1}^{T} s_c^{(t)}, \tag{41}$$

and the corresponding normalized firing rate is:

$$\hat{p}_c = \frac{1}{T} \sum_{t=1}^{T} s_c^{(t)}. \tag{42}$$

This firing rate $\hat{p}_c$ serves as an estimate of the probability that the network assigns to class c. It reflects the temporal frequency of spiking activity for each output neuron, under the assumption that neurons associated with the correct class should spike more frequently than others.

To train the network, we apply the standard cross-entropy loss between the predicted spike rates $\hat{p}_c$ and the ground-truth class label vector $\mathbf{y} = [y_1, y_2, \ldots, y_C]$, where $y_c = 1$ if class c is the true label and 0 otherwise. The loss is computed as:

$$L = -\sum_{c=1}^{C} y_c \log(\hat{p}_c) = -\sum_{c=1}^{C} y_c \log\left(\frac{1}{T} \sum_{t=1}^{T} s_c^{(t)}\right). \tag{43}$$

This formulation enables gradient-based optimization through the surrogate spike derivatives introduced in earlier sections. It effectively treats the average spike rate over time as a continuous proxy for probabilistic class scores, allowing the network to learn via backpropagation while preserving spike-based temporal encoding.

The use of rate-based decoding for classification is computationally simple and biologically plausible, and has been shown to yield competitive performance in many supervised SNN tasks. However, it also introduces a trade-off between accuracy and temporal resolution, as longer observation windows (larger T) are often required to obtain reliable estimates of spike rates.

6) Comparison with Alternative Training Paradigms: While surrogate gradient descent combined with Backpropagation Through Time (BPTT) has emerged as the most effective and scalable framework for training spiking neural networks (SNNs), several alternative training strategies have been explored in the literature. These methods vary in their biological plausibility, computational efficiency, and compatibility with modern machine learning frameworks. Below, we summarize and contrast some of the most notable approaches.

- **Spike-Timing Dependent Plasticity (STDP):**
  STDP is a biologically inspired, unsupervised learning rule that adjusts synaptic weights based on the relative timing of presynaptic and postsynaptic spikes. Typically, if a presynaptic spike precedes a postsynaptic spike within a short temporal window, the synapse is strengthened (long-term potentiation); if the order is reversed, the synapse is weakened (long-term depression). This learning mechanism is highly local and does not require error signals or global supervision. Although STDP has shown success in learning simple patterns, feature extraction, and early sensory processing, it lacks the scalability and task specificity required for training deep architectures on complex supervised objectives. Its limited performance in hierarchical, high-dimensional tasks makes it less suitable for modern SNN-based classification pipelines.
- **Random Feedback Alignment (RFA):**
  RFA is a biologically plausible alternative to traditional backpropagation, in which feedback weights used during gradient computation are replaced with fixed random matrices, rather than being symmetric with the feedforward weights. This approach mitigates the biological implausibility of exact weight transport, a long-standing criticism of standard backpropagation. While RFA has been shown to permit learning in both ANNs and SNNs to some extent, it significantly reduces the precision and stability of gradient updates, especially in deep or tightly constrained networks. Consequently, its performance often lags behind that of fully trained surrogate gradient methods.
- **Event-Driven Local Learning Rules:**
  In hardware-constrained or ultra-low-power systems, event-driven learning rules are often used. These include variants of STDP, Hebbian-like plasticity, and threshold-based adaptations that operate purely on local spike information without the need for global error signals or full forward-backward passes. While these methods are highly energy-efficient and hardware-friendly, they generally lack support for optimizing global objectives such as classification accuracy or structured representations. As a result, their applicability is typically limited to neuromorphic hardware demonstrations or early-stage sensory encoding.

In comparison, surrogate gradient learning with BPTT provides a compelling balance between biological inspiration and engineering utility. It enables training of deep, multi-layer SNNs using supervised losses and gradient-based optimization, all while retaining the key temporal and spiking properties of neural dynamics. Furthermore, it integrates seamlessly with modern deep learning infrastructure, including GPU acceleration and automatic differentiation frameworks, making it the most versatile and widely adopted training strategy for large-scale spiking models [1].

7) Training Challenges and Optimization Strategies: Despite the success of surrogate gradient methods, several practical challenges remain when training spiking neural networks (SNNs) via gradient descent. These challenges arise due to the unique characteristics of spiking models: discrete outputs, temporal dynamics, and sparse activation. Below, we discuss the key limitations encountered in practice and describe optimization strategies to address them.

- **Vanishing Gradients in Surrogate Approximation:**

  One of the most common issues in training deep SNNs is vanishing gradients, particularly when using surrogate derivatives that are too narrow or flat. Recall that the surrogate derivative $\sigma'(U)$ often takes the form of a scaled sigmoid function or another smooth bump around the spiking threshold θ:

$$\sigma'(U) = \frac{\gamma}{\left(1 + e^{-\beta(U-\theta)}\right)^2}, \tag{44}$$

  where γ controls the overall scale and β determines the slope near the threshold. If γ or β are set too low, then $\sigma'(U)$ becomes too flat, leading to near-zero gradients across a wide range of U. This causes slow learning or complete stagnation in weight updates, especially in deeper layers where gradient signals diminish over time. To mitigate this, β is often tuned empirically (e.g., $\beta = 10$–20), and γ is typically chosen to preserve gradient flow without destabilizing training. In addition, some approaches replace the sigmoid-based surrogate with higher-gradient alternatives such as piecewise linear or arctangent derivatives, which exhibit sharper transitions and wider support.
- **Inactive or "Dead" Neurons:**

  A common issue in both ANNs and SNNs is that some neurons may become permanently inactive — i.e., they never reach the spiking threshold and thus emit no spikes during training. Since the surrogate gradient is typically zero far from threshold, these neurons contribute no error signal and receive no meaningful updates. This is particularly problematic in early layers, where small membrane fluctuations may never accumulate enough to trigger spikes.
  One solution is to apply spike-based dropout, where a random subset of neurons is silenced at each timestep during training. This prevents any single subset of neurons from dominating the representation and encourages broader network participation. Another effective approach is activity regularization, which explicitly encourages firing by adding a regularization loss:

$$L_{\text{act}} = \lambda \sum_{i,t} \left| s_i^{(t)} - r_{\text{target}} \right|, \tag{45}$$

  where $s_i^{(t)}$ is the spike output of neuron i at time t, r_{target} is a desired firing rate (e.g., 0.1), and λ is a weighting term. This loss penalizes neurons that are too silent or overactive, pushing them toward a balanced spiking behavior.
- **Temporal Credit Assignment and Long-Term Dependencies:**

  As in recurrent neural networks (RNNs), spiking networks rely on temporal memory to solve sequential tasks. However, training over long sequences using Backpropagation Through Time (BPTT) becomes increasingly difficult due to the diminishing influence of earlier states. The gradient of a loss L at final timestep T with respect to parameters at timestep $t \ll T$ involves a product of many partial derivatives:

$$\frac{\partial L}{\partial \theta_t} = \frac{\partial L}{\partial U_T} \cdot \prod_{\tau=t}^{T} \frac{\partial U_\tau}{\partial U_{\tau-1}} \cdot \frac{\partial U_t}{\partial \theta_t}, \tag{46}$$

  where the product of $\frac{\partial U_\tau}{\partial U_{\tau-1}} \approx \alpha$ accumulates exponentially over time. Since $\alpha < 1$, this leads to gradients that decay exponentially with temporal distance — the hallmark of vanishing gradients in time.
  To alleviate this, several architectural modifications can be employed:
  - *Recurrent connections*, such as in Liquid State Machines or recurrent SNNs, introduce feedback loops that extend temporal memory.
  - *Skip connections* allow gradients to bypass intermediate layers or timesteps, reducing the depth of the unrolled computational graph.
  - *Spike accumulation* (e.g., counting spikes over blocks of time) allows the loss to depend on temporally aggregated behavior, which reduces sensitivity to precise timing and mitigates temporal vanishing gradients.

Collectively, these optimization strategies help improve gradient flow, promote balanced spiking activity, and extend the effective temporal receptive field of the network. They are essential for scaling surrogate gradient training to deeper and more temporally expressive SNN architectures.

8) Biological Plausibility and Theoretical Trade-offs: One of the central motivations behind spiking neural networks (SNNs) is their biological plausibility — the ability to emulate aspects of neural computation observed in the brain, such as event-driven signaling, temporal sparsity, and membrane dynamics. However, training methods based on surrogate gradients and backpropagation through time (BPTT) introduce mechanisms that deviate from strict biological realism.

In particular, gradient backpropagation requires non-local information flow, weight symmetry between forward and backward paths, and the storage of temporal traces — none of which are directly supported by known biological mechanisms. The brain does not appear to propagate error signals backward through synaptic connections in the precise, differentiable manner required by BPTT.

Despite these limitations, surrogate-gradient training can still be interpreted as a functional approximation to certain biologically plausible learning principles. For example, theories such as predictive coding and local dendritic error learning suggest that neurons may maintain estimates of prediction error and adjust synapses accordingly. Furthermore, local learning rules based on spike timing, membrane integration, and activity regularization can be incorporated into surrogate-based frameworks to improve alignment with neuroscience.

Thus, while surrogate-gradient descent sacrifices some aspects of biological realism, it retains core computational motifs of neural systems — such as temporally sparse coding and voltage-based integration — while enabling effective large-scale training using modern hardware.

9) Conclusion: Surrogate gradient methods provide a principled and scalable solution for training deep spiking neural networks. By replacing the non-differentiable spike generation function with a smooth surrogate during the backward pass, these methods enable end-to-end supervised learning while preserving the spiking behavior and temporal structure of the forward dynamics.

Combined with backpropagation through time (BPTT), surrogate-gradient descent supports the training of multi-layer SNNs on complex sequence-based tasks using gradient-based optimization and automatic differentiation frameworks. This approach integrates naturally with common neural coding schemes such as rate and latency coding, and benefits from acceleration on modern GPUs.

In our work, surrogate-gradient training was instrumental in achieving high accuracy on speech emotion recognition tasks. It enabled the spiking architecture to perform on par with conventional convolutional models while preserving the energy efficiency and event-driven nature of spike-based computation. As such, it forms the foundation of our learning pipeline, bridging biologically inspired computing with state-of-the-art machine learning tools.

E. Implementation Details

The entire model is implemented in PyTorch with snnTorch. We use Adam optimizer (lr=1e-3), weight decay $1e{-}4$, and train for 100 epochs. All experiments run on an NVIDIA RTX 3090 GPU. Source code and pretrained models will be released for reproducibility.

F. Impact of Feature Representation

TABLE V
EFFECT OF FEATURE ENCODING ON SNN ACCURACY

Feature Type	Accuracy (%)
MFCC Mean	63.3
MFCC Time Series	60.3
Stacked MFCC + RMSE + ZCR	**71.9**

The choice of acoustic feature representation plays a crucial role in the overall performance of the SNN classifier. Using only the mean of Mel-Frequency Cepstral Coefficients (MFCC) over time compresses all temporal structure into a single vector, which results in moderate accuracy (63.3%) due to the loss of dynamic information.

By preserving the full MFCC time series, the network can access frame-wise spectral patterns. However, this representation (60.3% accuracy) introduces redundancy and noise across time, which may hinder learning under sparse spike-driven dynamics.

The highest performance (71.9%) is achieved when combining stacked MFCCs with additional low-level descriptors such as root-mean-square energy (RMSE) and zero-crossing rate (ZCR). This composite feature vector enriches the input with both spectral and temporal cues, improving discriminability across emotion classes. These results highlight the benefit of multi-channel, information-rich encodings for downstream spike-based learning.

VI. DISCUSSION

The experimental results demonstrate that the proposed CNN-to-SNN conversion framework achieves highly competitive performance in speech emotion recognition (SER), while offering the additional benefits of energy efficiency and temporal sparsity inherent to spiking neural networks. In this section, we reflect on the observed outcomes, their implications, and limitations of the approach.

A. Performance Analysis

The fine-tuned SNN model achieves an accuracy of 87.8%, which is within 0.5% of the original CNN performance. This minimal drop in accuracy illustrates the effectiveness of rate-coded spike encoding combined with surrogate-gradient learning. Notably, the SNN achieves this result with significantly fewer floating-point operations due to sparse spiking activity, which translates into energy-efficient computation ideal for real-time or embedded applications.

The confusion patterns across emotion classes are consistent with psychological studies. Emotions such as “happy” and “neutral” or “fearful” and “sad” tend to have overlapping prosodic features, leading to misclassifications even in human perception. The SNN model retains class-wise balance in F1-score, indicating robustness across all target categories.

B. Feature Representation Impact

The superiority of stacked features (MFCC + RMSE + ZCR) suggests that SER models benefit from multi-dimensional representations. MFCC captures spectral structure, RMSE correlates with energy, and ZCR reflects signal frequency content. The fusion of these features offers a more holistic view of emotional content, particularly valuable for spike-based processing where richness of information must compensate for discretization noise.

C. Spike Encoding Implications

Among the spike encoding methods evaluated, rate coding yields the best results in terms of stability and ease of integration with existing gradient-based training pipelines. While latency-based or Poisson encoding could theoretically offer greater temporal precision, their training complexity and sensitivity to input variations make them less suitable for general-purpose SER tasks, especially in data-constrained settings.

D. Limitations and Considerations

Despite the promising results, certain limitations exist:

- The current model does not employ biologically plausible learning rules such as STDP, limiting its relevance to neuroscientific modeling.

- Spike encoding is applied post-feature extraction, not directly from raw waveform, which may introduce preprocessing biases.
- Real-time latency and deployment on neuromorphic hardware (e.g., Loihi) are not evaluated and remain open for future exploration.

Nonetheless, the CNN-to-SNN framework lays a strong foundation for energy-efficient, biologically inspired SER applications and highlights several avenues for further research.

VII. Conclusion

In this work, we have proposed a biologically inspired framework for speech emotion recognition (SER) based on a hybrid CNN-to-SNN architecture. The model integrates conventional convolutional front-end processing with spike-based fully connected classification layers, enabling efficient information encoding while maintaining high classification accuracy. By leveraging surrogate gradient learning and rate-based spike encoding, we successfully adapted deep learning optimization techniques to the non-differentiable nature of spiking neurons, thereby addressing one of the key challenges in training SNNs.

We also demonstrated that combining handcrafted low-level descriptors—such as MFCCs, RMSE, and ZCR—into a unified input representation significantly enhances emotion recognition performance. Our experiments across cleaned and harmonized datasets confirmed that the proposed model achieves competitive accuracy with traditional CNNs, while providing the additional advantage of sparse, event-driven computation, which is favorable for deployment in low-power neuromorphic hardware.

The CNN-to-SNN conversion approach offers a practical training pipeline by first learning with continuous activations and later transferring to a spiking regime with minimal loss. Moreover, surrogate gradient fine-tuning allows further improvement by directly optimizing within the spike domain.

Looking forward, several directions are worth exploring. These include developing adaptive and learnable spike encoding strategies, integrating temporal attention mechanisms in the spike domain, and extending the model to handle multimodal inputs such as facial expressions or physiological signals. Another important avenue is to investigate online learning and continual adaptation in dynamic real-world environments using neuromorphic principles.

Overall, this study highlights the viability of combining biologically plausible computation with modern machine learning frameworks to build scalable, interpretable, and energy-efficient emotion recognition systems.

References

[1] E. Neftci, H. Mostafa, and F. Zenke, "Surrogate Gradient Learning in Spiking Neural Networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 61–63, Nov. 2019. Available: https://arxiv.org/abs/2109.12894v6

[2] T. Rueckauer et al., "Conversion of Continuous-Valued Deep Networks to Efficient Event-Driven Networks for Image Classification," *Frontiers in Neuroscience*, vol. 11, p. 682, 2017.

[3] S. Sengupta et al., "Going Deeper in Spiking Neural Networks: VGG and Residual Architectures," *Frontiers in Neuroscience*, vol. 13, p. 95, 2019.

[4] F. Eyben, M. Wöllmer, and B. Schuller, "openSMILE – The Munich Versatile and Fast Open-Source Audio Feature Extractor," *Proc. ACM Multimedia*, 2010.

[5] D. M. Trigeorgis et al., "Adieu features? End-to-end speech emotion recognition using a deep convolutional recurrent network," *ICASSP*, 2016.

[6] S. Mirsamadi, E. Barsoum, and C. Zhang, "Automatic Speech Emotion Recognition Using Recurrent Neural Networks with Local Attention," *ICASSP*, 2017.

[7] R. Panda and K. Roy, "Unsupervised Regenerative Learning of Hierarchical Features in Spiking Deep Networks for Object Recognition," *Proc. IJCNN*, 2016.

[8] A. L. Hodgkin and A. F. Huxley, "A Quantitative Description of Membrane Current and Its Application to Conduction and Excitation in Nerve," *J. Physiol.*, vol. 117, pp. 500–544, 1952.

[9] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-Temporal Backpropagation for Training High-Performance Spiking Neural Networks," *Front. Neurosci.*, vol. 12, p. 331, 2018.

[10] B. Zenke and S. Ganguli, "Superspike: Supervised Learning in Multilayer Spiking Neural Networks," *Neural Computation*, vol. 30, no. 6, pp. 1514–1541, 2018.